J

PROYECTO PRÁCTICO

Manual de usuario y documentación técnica

Módulo de tareas · Verdulería Jerusalén

React Native	Expo SDK 58	TypeScript	AsyncStorage

Documento preparado para la demostración en Expo Go, Android y Expo Web.
Versión verificada: 10 de noviembre de 2026

1. Alcance de la entrega

El módulo de tareas permite organizar actividades operativas de la verdulería sin depender de una conexión permanente. La información se conserva localmente en el dispositivo y la misma base de código funciona en Android y en navegador.

CREAR	Registra título, descripción y categoría.	**CONSULTAR**	Muestra todas las tareas y sus contadores.
EDITAR	Carga una tarea existente y guarda cambios.	**ELIMINAR**	Solicita confirmación antes de borrar.

Funciones incluidas

- CRUD completo: crear, visualizar, editar y eliminar tareas.
- Cambio rápido entre pendiente y completada.
- Filtros: Todas, Pendientes y Completadas.
- Contadores que se actualizan con cada operación.
- Persistencia automática con serialización JSON.
- Interfaz adaptable para teléfono, tableta y computadora.

Nota de seguridad

AsyncStorage es almacenamiento persistente local no cifrado. En este proyecto solo guarda tareas operativas, no contraseñas ni información sensible.

Tecnologías verificadas

Tecnología	Uso	Estado
React Native + Expo SDK 58	Aplicación multiplataforma	Compatible
TypeScript estricto	Modelos y funciones tipadas	Sin errores
AsyncStorage	Persistencia JSON local	Implementado

2. Guía de uso

La pantalla se abre desde el menú administrativo en Tareas. En teléfono, el menú lateral se abre con el botón superior izquierdo.



1. Revise los contadores

La cabecera resume tareas totales, pendientes y completadas.

2. Cree una tarea

Escriba un título de al menos tres caracteres, agregue una indicación opcional, elija Inventario, Pedidos o General y presione Agregar tarea.

3. Filtre la lista

Use Todas, Pendientes o Completadas para encontrar rápidamente la actividad necesaria.

4. Trabaje sin conexión

Las tareas quedan en el almacenamiento del dispositivo. Al volver a abrir la aplicación, se recuperan automáticamente.

3. Operaciones CRUD

Crear

- Complete el título; la descripción es opcional.
- Seleccione una categoría y presione Agregar tarea.
- La nueva tarea aparece primero y queda marcada como pendiente.

Editar

- Presione Editar en la tarjeta de la tarea.
- El formulario carga los valores actuales. Modifique y presione Guardar cambios.
- Use Cancelar para salir sin alterar la tarea.

Completar o reabrir

- Presione el control de estado. La tarea cambia entre pendiente y completada y se actualizan los contadores.

Eliminar

- Presione Eliminar. El sistema solicita confirmación para evitar borrados accidentales.
- Al confirmar, el registro se elimina del estado y de AsyncStorage.

Reglas y mensajes

Situación	Respuesta del sistema
Título menor de 3 caracteres	Muestra una validación y no guarda.
Filtro sin resultados	Presenta un estado vacío con orientación.
Error al leer almacenamiento	Muestra un aviso accesible; la pantalla sigue disponible.
Error al guardar	Informa que el cambio no pudo conservarse en el dispositivo.

Comprobación de persistencia

Cree una tarea, cierre completamente la aplicación o recargue el navegador y vuelva a Tareas. El registro debe conservarse. En web puede revisar las herramientas del navegador (F12 → Application/Aplicación → almacenamiento del sitio).

4. Arquitectura y organización

La pantalla está separada por responsabilidades para facilitar mantenimiento, pruebas y reutilización.

```
frontend/src/
├─ app/tareas.tsx # composición de la pantalla
├─ types/todo.ts # Task, TaskDraft y filtros
├─ services/todoStorage.ts # lectura/escritura de AsyncStorage
├─ hooks/useTodoList.ts # estado, CRUD y filtros
└─ components/
├─ TaskInput.tsx # formulario y validación
├─ TaskItem.tsx # tarjeta y acciones
└─ TaskFilter.tsx # filtros de visualización
```

types/todo.ts	Define contratos estrictos sin any.
todoStorage.ts	Aísla la clave y la serialización JSON.
useTodoList.ts	Centraliza estado, CRUD, filtros y estadísticas.
Componentes	Separan formulario, tarjeta y filtros.

Modelo principal

```
export interface Task {
id: string;
title: string;
description: string;
category: TaskCategory;
completed: boolean;
createdAt: string;
updatedAt: string;
}
```

Renderizado eficiente

La lista usa FlatList con keyExtractor, de modo que React Native identifica cada elemento por su id y virtualiza el contenido. En la versión ajustada la lista tiene su propio desplazamiento y no se desactiva la virtualización.

5. Persistencia con AsyncStorage

Cada operación genera el arreglo actualizado y lo guarda bajo una clave exclusiva. Al montar el hook, la aplicación recupera el JSON, lo analiza y valida los campos mínimos antes de mostrarlo.

Servicio de almacenamiento

```
const STORAGE_KEY = '@verduleria_tareas_v1';

async saveTasks(tasks: Task[]): Promise<void> {
await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

async getTasks(): Promise<Task[]> {
const stored = await AsyncStorage.getItem(STORAGE_KEY);
if (!stored) return [];
const parsed: unknown = JSON.parse(stored);
return Array.isArray(parsed) ? parsed.filter(isTask) : [];
}
```

Flujo de datos

1	El usuario ejecuta una acción en TaskInput o TaskItem.
2	useTodoList calcula el nuevo arreglo de tareas.
3	persist actualiza la interfaz y llama a todoStorage.saveTasks.
4	AsyncStorage conserva el JSON localmente.
5	En la siguiente apertura, getTasks restaura los datos.

Manejo de errores

La lectura y escritura se ejecutan con promesas controladas. Si ocurre un fallo, el hook usa storageError para mostrar un mensaje visible y accesible sin cerrar la aplicación.

Depuración recomendada

- Confirme que @react-native-async-storage/async-storage esté instalado con Expo.
- Revise la consola del navegador o Metro si aparece el aviso de almacenamiento.
- No edite manualmente el JSON de producción; use tareas de prueba.

6. Instalación, ejecución y pruebas

Requisitos

- Node.js y npm disponibles.
- Expo Go actualizado para probar en teléfono Android, o un navegador moderno para web.

Comandos

```
cd frontend
npm install
npx expo start # abre el servidor de desarrollo
npm run web # ejecuta Expo Web
npm run check # TypeScript + ESLint + Expo Doctor + export web
```

Prueba en Android con Expo Go

- Conecte computadora y teléfono a la misma red.
- Ejecute npx expo start y escanee el código QR con Expo Go.
- Inicie sesión, abra Tareas y realice el ciclo crear → editar → completar → eliminar.

Resultados de verificación

Control	Resultado	Evidencia
TypeScript	APROBADO	npx tsc --noEmit: 0 errores
ESLint	APROBADO	npm run lint: sin errores
Expo Doctor	APROBADO	20/20 comprobaciones
Expo Web	APROBADO	Exportación web completada
Android	APROBADO	Bundle Hermes generado

Prueba funcional sugerida

Cree tres tareas de diferentes categorías; edite una, complete otra y elimine la tercera. Cambie entre los tres filtros, cierre la aplicación y vuelva a abrirla. Los datos y contadores deben coincidir.

7. Matriz de cumplimiento

La siguiente tabla relaciona la guía de evaluación con la evidencia disponible en la entrega.

Criterio	Implementación / evidencia	Valor
CRUD	Crear, visualizar, editar, completar/reabrir y eliminar con confirmación.	5 pts
Persistencia	AsyncStorage, clave propia, JSON.stringify/parse, carga inicial y mensajes de error.	4 pts
Arquitectura y TypeScript	Tipos, servicio, hook y tres componentes separados; compilación estricta sin errores.	3 pts
Interfaz y experiencia	Pressable, FlatList con keyExtractor, SafeAreaProvider/SafeAreaView, filtros, estados vacíos y diseño adaptable.	3 pts
Manual y depuración	Documento PDF con guía, arquitectura, código, almacenamiento, pruebas y capturas.	5 pts

Lista de comprobación para entregar

- PDF del manual incluido.
- Código fuente organizado y sin archivos de credenciales.
- Paquete ZIP generado desde la versión validada.
- Aplicación verificable en web y Android/Expo Go.
- Demostración preparada: crear, persistir, filtrar, editar, completar y eliminar.

ENTREGA LISTA — **El módulo cumple los puntos funcionales, de persistencia, arquitectura, interfaz y documentación solicitados.**